

Lab Manual #08

Structured Query Language (SQL)

SQL Joins

SQL Joins are used when you need to consolidate data from two different tables based on a common attribute or that have a relationship between them. This can be a one-to-one, one-to-many, or many-to-many relation.

The JOIN clause allows you to create a logical relationship between two or more tables. See the example below for further understanding:

Students Table:

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	JOHN	LONDON	XXXXXXXXXX	18
2	ALEX	MANCHESTER	XXXXXXXXXX	19
3	EMMA	BRISTOL	XXXXXXXXXX	20
4	LUCAS	OXFORD	XXXXXXXXXX	18
5	SOPHIA	LIVERPOOL	XXXXXXXXXX	19
6	DAVID	BIRMINGHAM	XXXXXXXXXX	20
7	JAMES	CAMBRIDGE	XXXXXXXXXX	18
8	HENRY	YORK	XXXXXXXXXX	19

StudentCourse Table:

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

JOIN Query:

```
SELECT s.roll_no, s.name, s.address, s.phone, s.age, sc.course_id
FROM Student s
JOIN StudentCourse sc ON s.roll_no = sc.roll_no;
```

Output:

ROLL_NO	NAME	ADDRESS	PHONE	AGE	COURSE_ID
1	JOHN	LONDON	XXXXXXXXXX	18	1
2	ALEX	MANCHESTER	XXXXXXXXXX	19	2
3	EMMA	BRISTOL	XXXXXXXXXX	20	2
4	LUCAS	OXFORD	XXXXXXXXXX	18	3
5	SOPHIA	LIVERPOOL	XXXXXXXXXX	19	1

As you can see in the provided example, the data from both the tables are consolidated in the output based on the attribute/field 'roll_no'.

Types of SQL Joins

Now there are many types of JOINS that can be used upon different condition. We will be studying the following:

1. Inner JOIN
2. Left JOIN
3. Right JOIN
4. Full Outer JOIN
5. Self JOIN
6. Cross JOIN
7. Natural JOIN

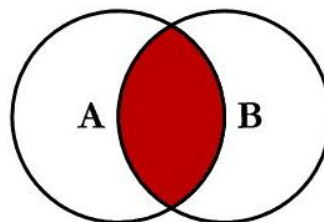
1. Inner JOIN

The INNER JOIN keyword selects records that have matching values in both tables. By default, a join clause is set to INNER JOIN, so even if you omit the INNER keyword the JOIN clause would work the same.

Syntax:

```
SELECT column_name(s) FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name;
```

A visual representation of the INNER JOIN is shown below:



```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```

For the same table of Student and StudentCourse, the provided code below would give the following output:

Code:

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM Student  
INNER JOIN StudentCourse  
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Output:

COURSE_ID	NAME	Age
1	JOHN	18
2	ALEX	19
2	EMMA	20
3	LUCAS	18
1	SOPHIA	19

2. Left JOIN

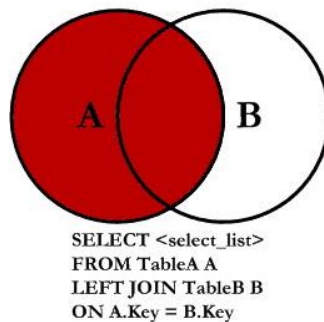
The LEFT JOIN keyword returns all records from the left table (table1), and the matched records from the right table (table2). The result is NULL from the right side, if there is no match. (Inner Join+ rest of records from left table)

LEFT JOIN is a subset of the OUTER JOIN joins.

Syntax:

```
SELECT column_name(s) FROM table1 LEFT JOIN table2 ON table1.column_name = table2.column_name;
```

A visual representation of the LEFT JOIN is shown below:



For the same table of Student and StudentCourse, the provided code below would give the following output:

Code:

```
SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

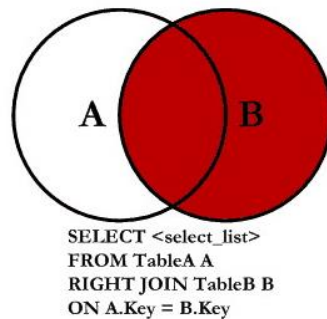
Output:

NAME	COURSE_ID
JOHN	1
ALEX	2
EMMA	2
LUCAS	3
SOPHIA	1
DAVID	NULL
JAMES	NULL
HENRY	NULL

3. Right JOIN

Similar to LEFT JOIN, RIGHT JOIN returns all the records on the right side of the join and the corresponding records from the left side (Inner Join+ rest of records from right table), even if the left side has no said values, in which case NULL values will be returned. RIGHT JOIN is a subset of the OUTER JOIN joins.

A visual representation of the RIGHT JOIN is shown below:



For the same table of Student and StudentCourse, the provided code below would give the following output:

Code:

```
SELECT Student.NAME, StudentCourse.COURSE_ID  
FROM Student  
RIGHT JOIN StudentCourse  
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

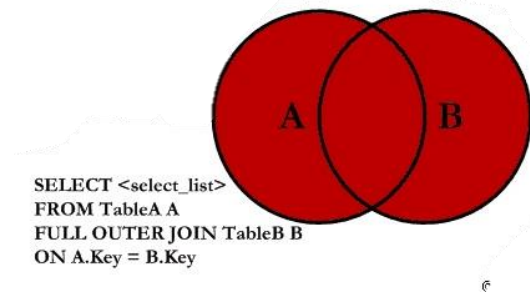
Output:

NAME	COURSE_ID
JOHN	1
ALEX	2
EMMA	2
LUCAS	3
SOPHIA	1
NULL	4
NULL	5
NULL	4

4. Full Outer JOIN

FULL OUTER JOIN returns all the records/rows from both the tables even if their corresponding values from the other table have no values/NULL. You can think of it as a combination of the LEFT JOIN and the RIGHT JOIN.

A visual representation of the FULL OUTER JOIN is shown below:



For the same table of Student and StudentCourse, the provided code below would give the following output:

Code:

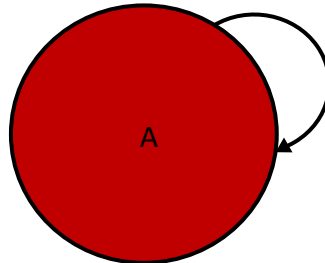
```
SELECT Student.NAME, StudentCourse.COURSE_ID  
FROM Student  
FULL JOIN StudentCourse  
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
JOHN	1
ALEX	2
EMMA	2
LUCAS	3
SOPHIA	1
DAVID	NULL
JAMES	NULL
HENRY	NULL
NULL	4
NULL	5
NULL	4

5. Self JOIN

SELF JOIN clause is a special clause where the table is joined to itself, meaning that each row/record of the table is combined with itself. To help visualize this, you can think of the same table as having two similar copies. You can further help yourself with uses aliases for the tables allowing you to the temporary table names(alias) in the queries and not cause confusion.



For the Student Table Below, the SELF JOIN would work as shown in the example:
Table:

id	admission_no	first_name	last_name	age	city
1	3354	Luisa	Evans	13	Texas
2	2135	Paul	Ward	15	Alaska
3	4321	Peter	Bennett	14	California
4	4213	Carlos	Patterson	17	New York
5	5112	Rose	Huges	16	Florida
6	6113	Marielia	Simmons	15	Arizona
7	7555	Antonio	Butler	14	New York
8	8345	Diego	Cox	13	California

Code:

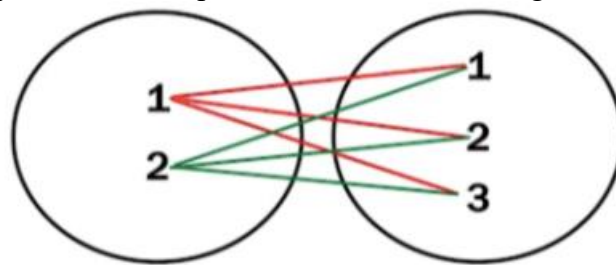
```
SELECT S1.first_name, S2.last_name, S2.city
FROM Student S1
join Student S2
WHERE S1.id <> S2.iD AND S1.city = S2.city
ORDER BY S2.city;
```

Output:

first_name	last_name	City
Peter	Cox	California
Diego	Bennett	California
Carlos	Butler	New York
Antonio	Patterson	New York

6. Cross JOIN

CROSS JOIN or also known as CARTESIAN JOIN returns all the possible combinations of the tables joined together, hence returning every row from each table. The combination of the tables is done similarly to a cartesian product, as seen in the diagram below:



The following illustrates syntax of the CROSS JOIN clause:

```
SELECT column_list  
FROM A  
CROSS JOIN B;Code language:
```

The following picture illustrates the result of the cross join between the table A and table B. In this illustration, the table A has three rows 1, 2 and 3 and the table B also has three rows x, y and z. As the result, the Cartesian product has nine rows:

A	B		A x B
n	c		n c
1	x	SELECT * FROM A CROSS JOIN B	1 x
2	y		1 y
3	z		1 z
			2 x
			2 y
			2 z
			3 x
			3 y
			3 z

Note that unlike the [INNER JOIN](#), [LEFT JOIN](#), the CROSS JOIN clause does not have a join condition.

SELECT

column_list

FROM

A CROSS JOIN B;

To find the all possible sales channels that a sales organization can have, you use the CROSS JOIN to join the sales_organization table with the sales_channel table as follows:

SELECT

sales_org,

channel

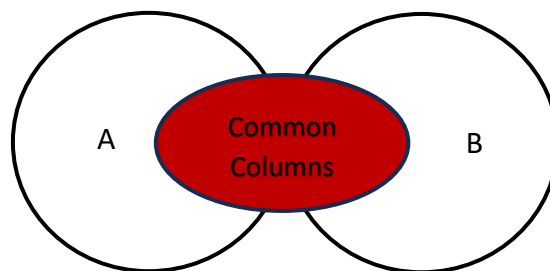
FROM

sales_organization

CROSS JOIN sales_channel;

7. Natural JOIN

NATURAL JOIN clause joins the tables based on the common field/column in the two or more tables being joined. The field must have both similar name and data type and be present in both tables. This condition must be true for at least one field for you to be allowed to use NATURAL JOIN. Also, a thing to note is the when using NATURAL JOIN, unlike in INNER JOIN, common rows/duplicates aren't returned in NATURAL JOIN.



Below is the Syntac for NATURAL JOIN along with an example for the tables ‘employee’ and ‘departments’ respectively:

Tables:

EMP_ID	EMP_NAME	DEPT_NAME
1	JOHN	HR
2	ALEX	IT
3	EMMA	MARKETING
4	LUCAS	IT
5	SOPHIA	SALES

DEPT_NAME	MANAGER_NAME
IT	DAVID
SALES	JAMES
HR	HENRY
FINANCE	GEORGE
MARKETING	CHARLES

Code:

```
SELECT *
FROM employee
NATURAL JOIN departments
```

Output:

EMP_ID	EMP_NAME	DEPT_NAME	MANAGER_NAME
1	JOHN	HR	HENRY
2	ALEX	IT	DAVID
3	EMMA	MARKETING	CHARLES
4	LUCAS	IT	DAVID
5	SOPHIA	SALES	JAMES